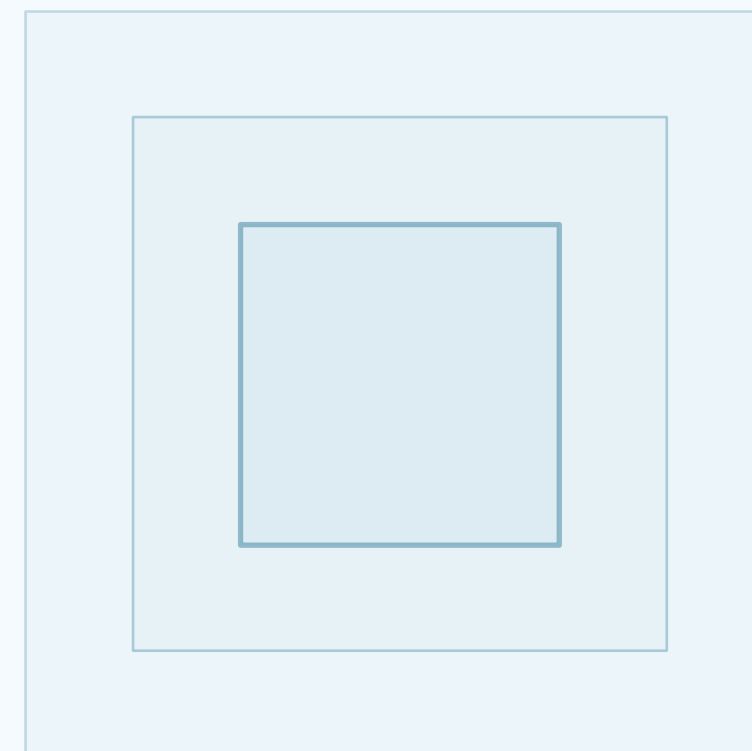


# Colecciones y Funciones en Python

LISTAS · TUPLAS · DICCIONARIOS · FUNCIONES

- ◆ **Docente: Taller de Programación 1**
- ◆ Facultad de Ingeniería y Tecnología
- ◆ Universidad San Sebastián · Campus Bellavista
- ◆ 2026





# Contenidos

CONTENTS

CONTENT

## ◆ Colecciones en Python

Listas, Tuplas y Diccionarios

---

## ◆ Funciones en Python

Definición, Tipos, Parámetros y Buenas Prácticas

---

## ◆ Ejemplos Prácticos Modernos

Casos Reales para el Desarrollo Profesional

# 01

## CAPÍTULO UNO

# Colecciones en Python

Listas, Tuplas y Diccionarios

---

1.1 Listas y sus operaciones

1.2 Tuplas e inmutabilidad

1.3 Diccionarios y pares clave-valor

## ¿Qué son las Colecciones?

### ◆ Estructuras de datos fundamentales

Las colecciones permiten almacenar, organizar y manipular grupos de valores en una sola variable. Son esenciales para cualquier programa que maneje múltiples datos relacionados.

### ◆ Tres colecciones principales

**Listas** (mutables, ordenadas), **Tuplas** (inmutables, ordenadas) y **Diccionarios** (pares clave-valor). Cada una tiene características específicas para diferentes escenarios.

### ◆ ¿Por qué son importantes?

Representan datos del mundo real: una lista de reproducción, coordenadas GPS, el perfil de un usuario. Dominarlas es clave para desarrollar aplicaciones profesionales.

### ◆ Ejemplo rápido

```
lista = ["Python", "JavaScript", "Rust"] # Lista mutable | tupla = (40.4168, -3.7038) # Tupla inmutable (coordenadas) | dict = {"nombre": "Ana", "edad": 21}
# Diccionario clave-valor
```

◆ Clave: elegir la colección correcta según si los datos cambian, su orden y cómo necesitas acceder a ellos.

## Listas en Python

### Características principales

- **Ordenadas:** los elementos mantienen su posición
- **Mutables:** se pueden modificar después de crearlas
- **Permiten duplicados:** valores repetidos son válidos
- **Heterogéneas:** pueden contener diferentes tipos
- Se definen con corchetes: `[ ]`

### Ejemplos modernos y significativos

- Feed de publicaciones (Instagram, TikTok)
- Lista de reproducción (Spotify, YouTube)
- Carrito de compras (Amazon, MercadoLibre)
- Historial de navegación del navegador
- Mensajes de chat (WhatsApp, Discord)

### Ejemplo de código

```
# Playlist de música  
playlist = ["Blinding Lights", "Levitating", "Peaches"]  
playlist.append("As It Was") # Agregar al final  
playlist.insert(0, "Anti-Hero") # Insertar al inicio  
print(playlist[0]) # → "Anti-Hero" (indexación)
```

**Tip:** Las listas son la colección más flexible de Python. Úsalas cuando necesites una secuencia de elementos que puede cambiar con el tiempo.

## Operaciones y Métodos de Listas

| Método      | Descripción                | Ejemplo           |
|-------------|----------------------------|-------------------|
| append(x)   | Agrega x al final          | lista.append(5)   |
| insert(i,x) | Inserta x en índice i      | lista.insert(0,a) |
| remove(x)   | Elimina primera x          | lista.remove(3)   |
| pop(i)      | Elimina y retorna índice i | lista.pop()       |
| sort()      | Ordena ascendente          | lista.sort()      |
| reverse()   | Invierte el orden          | lista.reverse()   |
| clear()     | Elimina todos              | lista.clear()     |

### Ejemplo: procesar datos

```
# Filtrar productos por precio
productos = [
    {"nombre": "Laptop", "precio": 899},
    {"nombre": "Mouse", "precio": 25},
    {"nombre": "Monitor", "precio": 299}
]

# List comprehension moderna
caros = [p for p in productos
        if p["precio"] > 200]

print(caros) # Laptop y Monitor
```

◆ List Comprehension: forma concisa y eficiente de crear listas. [expr for item in iterable if condicion]

### Slicing: acceso avanzado a sublistas

◆ lista[inicio:fin:paso] — Extrae una porción de la lista. lista[0:3] primeros 3 elementos | lista[-3:] últimos 3 | lista[::-1] lista invertida | lista[::2] cada 2 elementos



# Tuplas en Python

01 Introducción

02 Colecciones

03 Funciones

04 Ejemplos

05 Cierre

## Características de las Tuplas

- ◆ **Inmutables**: no se pueden modificar después de crearlas
- ◆ **Ordenadas**: mantienen el orden de inserción
- ◆ **Permiten duplicados**
- ◆ Se definen con paréntesis: ( )
- ◆ Más rápidas y eficientes que las listas
- ◆ Son **hashables**: pueden usarse como claves de diccionario
- ◆ Ideales para datos que no deben cambiar

## Ejemplos modernos significativos

**Coordenadas GPS**  
`ubicacion = (-33.4489, -70.6693)`

**Fecha como tupla**  
`fecha = (2026, 4, 27) # año, mes, día`

**Retorno múltiple de función**  
`def min_max(numeros):`  
`return min(numeros), max(numeros)`  
`minimo, maximo = min_max([3, 1, 4, 1, 5])`

**Desempaquetado**  
`nombre, edad, ciudad = ("Ana", 21, "Santiago")`

## Listas vs Tuplas: ¿Cuándo usar cada una?

| Característica      | Lista [ ]                       | Tupla ( )                         |
|---------------------|---------------------------------|-----------------------------------|
| Mutabilidad         | Mutable (se puede modificar)    | Inmutable (no se puede modificar) |
| Rendimiento         | Más lenta, más memoria          | Más rápida, menos memoria         |
| Uso como clave      | No puede ser clave de dict      | Sí puede ser clave de dict        |
| Métodos disponibles | append, insert, remove, sort... | Solo count e index                |
| Sintaxis            | [1, 2, 3]                       | (1, 2, 3) o 1, 2, 3               |

### Guía de decisión práctica

#### Usa LISTAS cuando:

- Los datos cambian dinámicamente
- Carrito de compras, mensajes de chat
- Tareas pendientes, notificaciones
- Necesitas ordenar o filtrar frecuentemente

#### Usa TUPLAS cuando:

- Los datos no deben cambiar
- Coordenadas, fechas, configuración
- Retorno múltiple de funciones
- Necesitas usarla como clave de diccionario

**Regla mnemotécnica:** Si los datos "escriben" (cambian) → Lista. Si los datos "leen" (son fijos) → Tupla.



## Diccionarios en Python

### Conceptos fundamentales

- Almacenan pares **clave : valor**
- Acceso ultrarrápido  $O(1)$  por clave
- Claves únicas, valores duplicables
- Desde Python 3.7: mantienen orden
- Sintaxis: `{ clave: valor }`

### Usos modernos significativos

- APIs REST (formato JSON)
- Perfiles de usuario en apps
- Configuraciones de aplicaciones
- Cachés y memoización
- Bases de datos NoSQL (MongoDB)

### Ejemplo: perfil de usuario

```
usuario = {"nombre": "Camila", "edad": 22, "carrera": "Ing."}
print(usuario["nombre"]) # → "Camila"
usuario["semestre"] = 5 # Agregar nueva clave
for clave, valor in usuario.items():
    print(f"{clave}: {valor}") # Iterar con .items()
```

**Tip:** Los diccionarios son la estructura más versátil de Python. Úsalos cuando necesites acceder a datos por un identificador único (nombre, ID, email).

## Métodos y Operaciones con Diccionarios

| Método          | Descripción               | Ejemplo               |
|-----------------|---------------------------|-----------------------|
| get(k, default) | Obtiene valor o default   | d.get('x', 0)         |
| keys()          | Retorna todas las claves  | list(d.keys())        |
| values()        | Retorna todos los valores | list(d.values())      |
| items()         | Retorna pares (k, v)      | for k, v in d.items() |
| update(d2)      | Fusiona otro diccionario  | d1   d2 (Py 3.9+)     |
| pop(k)          | Elimina y retorna clave   | d.pop('x')            |

### Ejemplo: contador de palabras

```
texto = "python es genial y python es poderoso"
palabras = texto.split()
conteo = {}
for palabra in palabras:
    conteo[palabra] = conteo.get(palabra, 0) + 1
print(conteo)
# {'python': 2, 'es': 2, 'genial': 1, ...}
```



**Dictionary Comprehension** (forma concisa de crear diccionarios):

```
cuadrados = {x: x**2 for x in range(5)} # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

**Fusionar diccionarios** (Python 3.9+): `config = defaults | user_settings`

# 02

## CAPÍTULO DOS

# Funciones en Python

Definición, Tipos, Parámetros y Buenas Prácticas

---

2.1 Definición y estructura de una función

2.2 Tipos de funciones y parámetros

2.3 Ejemplos prácticos y buenas prácticas

## ¿Qué son las Funciones?

### ◆ Bloques de código reutilizables

Una función es un bloque de código que realiza una tarea específica y puede ser llamada múltiples veces desde diferentes partes del programa. Evitan repetir código y facilitan el mantenimiento.

### ◆ Principio DRY: Don't Repeat Yourself

Si necesitas ejecutar las mismas instrucciones más de una vez, encapsúlalas en una función. Esto reduce errores, facilita cambios y hace el código más legible y profesional.

### ◆ Componentes de una función

**def** (palabra clave) + **nombre** descriptivo + **parámetros** (entrada) + **cuerpo** (lógica) + **return** (salida opcional). Las funciones pueden recibir datos, procesarlos y devolver resultados.

```
def calcular_descuento(precio, porcentaje): # definición
    """Calcula el precio final con descuento aplicado.""" # docstring
    return precio * (1 - porcentaje / 100) # retorno
```

◆ Las funciones son la base de la programación modular: dividir para conquistar problemas complejos.

# Tipos de Funciones en Python

## Built-in

print(), len(), range(), map(), filter(), sorted()  
Integradas en Python

## Definidas por usuario

Creadas con def nombre():  
Personalizadas para tu lógica

## Lambda (anónimas)

lambda x: x \* 2  
Funciones de una línea, sin nombre

## Generadoras

Usan yield en lugar de return  
Producen valores perezosamente (ahorro de memoria)

## Rekursivas

Se llaman a sí mismas  
Factorial, Fibonacci, recorrido de árboles

## Orden Superior

Aceptan o retornan funciones  
Decoradores, callbacks, closures

## Ejemplos de cada tipo

```
# Lambda + orden superior (filter) # Generadora
pares = list(filter(lambda x: x % 2 == 0, range(10)))
def contador(n):
    for i in range(n):
        # Recursiva yield i
        def factorial(n):
            for num in contador(5):
                return 1 if n <= 1 else n * factorial(n-1)
        print(num) # 0, 1, 2, 3, 4
```

**Tip:** Comienza con funciones def básicas. Domina lambda y generadores para escribir código más Pythonico y eficiente.

# Parámetros y Argumentos

Tipos de Parámetros

**Posicionales:** obligatorios, en orden

```
def suma(a, b): ...
```

**Nombrados (keyword):** con nombre

```
suma(a=3, b=5)
```

**Con valor por defecto:**

```
def saludar(nombre="Usuario"): ...
```

**\*args:** argumentos variables (tupla)

**\*\*kwargs:** argumentos nombrados variables (dict)

Orden Obligatorio en Python

1. Parámetros **posicionales**
2. Parámetros **con default**
3. **\*args** (tupla de extras)
4. Parámetros **keyword-only**
5. **\*\*kwargs** (dict de extras)

```
def ejemplo(a, b=2, *args, c, **kwargs):  
    pass
```

Ejemplo moderno: configuración de API

```
def configurar_app(nombre, entorno="dev", *modulos, debug=False, **opciones_extra):  
    """Configura una app web con módulos y opciones."""  
    config = {"nombre": nombre, "entorno": entorno, "debug": debug, "modulos": modulos}  
    config.update(opciones_extra); return config
```

**Clave:** \*args y \*\*kwargs dan flexibilidad total. Úsalos cuando no sepas de antemano cuántos argumentos recibirá tu función.



# Ejemplos Prácticos Modernos



```
# 1: Procesar datos de API de clima (dict + list)
clima = {"ciudad": "Santiago", "temp": 28, "pronostico": [{"dia": "Lun", "t": 30}, {"dia": "Mar", "t": 27}]}
print(clima["pronostico"][0]["t"]) # 30

# 2: Sistema de gestión de tareas con filtros
def filtrar_tareas(tareas, estado): return [t for t in tareas if t["estado"] == estado]

# 3: Analizador de sentimientos básico
sentimientos = {"excelente": 1, "malo": -1, "regular": 0}
def score(texto): return sum(sentimientos.get(p, 0) for p in texto.split())
```

Estos ejemplos combinan colecciones y funciones de forma integral, tal como se usan en desarrollo profesional real.

# Buenas Prácticas y Consideraciones

01 Introducción

02 Colecciones

03 Funciones

04 Ejemplos

05 Cierre

## Principios Fundamentales

- ◆ **Una función = una responsabilidad** (SRP)
- ◆ Nombres descriptivos: verbo + sustantivo  
calcular\_total(), filtrar\_usuarios()
- ◆ **Documentar** con docstrings triple comilla
- ◆ Evitar efectos secundarios cuando sea posible
- ◆ Manejar excepciones con try/except
- ◆ Usar type hints para claridad
- ◆ Máximo 20-30 líneas por función
- ◆ Evitar variables globales
- ◆ Preferir composición sobre anidación

## Ejemplo: función bien escrita

```
def calcular_promedio_notas(
    notas: list[float]
) -> float:
    """
    Calcula el promedio de una lista de notas.
    Args: notas: Lista de calificaciones
    Returns: Promedio redondeado a 1 decimal
    Raises: ValueError si la lista está vacía
    """

    if not notas:
        raise ValueError("Lista vacía")
    return round(sum(notas) / len(notas), 1)
```

**Testing:** Verifica tus funciones con casos de prueba. Usa assert para validaciones simples o pytest para proyectos profesionales.

```
assert calcular_promedio_notas([6.5, 7.0, 5.5]) == 6.3
```



# ¡A programar!

PRACTICE MAKES PERFECT

◆ **Taller de Programación 1** | Universidad San Sebastián

◆ Facultad de Ingeniería y Tecnología

◆ La práctica constante es el camino para dominar Python

◆ 2026

¿Preguntas? | Q & A

